YummyLand

Application web e-commerce Jellycat

Spécifications techniques & fonctionnelles

Stack : Next.js 14 · Prisma · MySQL · NextAuth · Tailwind CSS

2025 — 2026

1. Présentation du projet

1.1 Contexte

YummyLand est une boutique en ligne dédiée aux produits Jellycat. L'application permet aux clients de naviguer dans le catalogue, d'ajouter des articles à leur panier et de passer des commandes. Un système d'administration permet de gérer les produits et de consulter la liste des clients.

1.2 Choix techniques

Technologie	Rôle
Next.js 14 (App Router)	Framework React — frontend + API routes
Prisma	ORM — accès à la base de données MySQL
MySQL	Base de données relationnelle
NextAuth.js	Authentification JWT (connexion / session)
Tailwind CSS	Framework CSS utilitaire — design rose
bcryptjs	Hashage sécurisé des mots de passe

2. Modélisation des données

2.1 Schéma relationnel

La base de données MySQL comporte 6 tables :

- User — les utilisateurs (clients + admins)
- Product — le catalogue de produits Jellycat
- Cart — le panier de chaque utilisateur (1 panier par user)
- CartItem — les articles dans un panier (produit + quantité)
- Order — les commandes passées
- OrderItem — les articles d'une commande (produit + quantité + prix au moment de l'achat)

2.2 Schéma Prisma

Table User

Champ	Type	Description
id	Int (PK, auto)	Identifiant unique auto-incrémenté
name	String	Prénom de l'utilisateur
email	String (unique)	Email unique — sert d'identifiant de connexion
password	String	Mot de passe hashé avec bcrypt
role	Enum Role	USER (défaut) ou ADMIN
createdAt	DateTime	Date de création du compte

Table Product

Champ	Type	Description
id	Int (PK, auto)	Identifiant unique
name	String	Nom du produit
description	String	Description affichée dans la boutique
price	Float	Prix en euros
image	String	Chemin vers l'image (/images/produit.jpg)
category	Enum Category	PELUCHE | PORTE_CLEF | SAC_A_DOS

Table Order

Champ	Type	Description
id	Int (PK, auto)	Identifiant unique
orderNumber	String (unique)	Numéro lisible — ex : YL-20250316-4821

userId	Int (FK)	Référence vers l'utilisateur
total	Float	Montant total de la commande en euros
status	Enum OrderStatus	EN_ATTENTE | EN_COURS | LIVREE
createdAt	DateTime	Date de la commande

3. Cas d'utilisation

3.1 Visiteur non connecté

Cas	Description	Page
UC-1	Voir la page d'accueil avec les 3 catégories	/
UC-2	Parcourir le catalogue et filtrer par catégorie	/shop
UC-3	S'inscrire avec email, prénom et mot de passe sécurisé	/register
UC-4	Se connecter avec email et mot de passe	/login

3.2 Visiteur connecté (CLIENT)

Cas	Description	Page
UC-5	Ajouter un produit au panier	/shop
UC-6	Consulter son panier	/cart
UC-7	Modifier la quantité d'un article dans le panier	/cart
UC-8	Supprimer un article du panier	/cart
UC-9	Passer une commande depuis le panier	/cart
UC-10	Consulter l'historique de ses commandes	/orders
UC-11	Modifier son prénom / email	/profile
UC-12	Modifier son mot de passe	/profile
UC-13	Se déconnecter	Navbar

3.3 Administrateur

Cas	Description	Page
UC-14	Ajouter un nouveau produit au catalogue	/admin
UC-15	Supprimer un produit du catalogue	/admin
UC-16	Consulter la liste des clients et leurs commandes	/admin

4. Routes API

4.1 Authentification

Méthode	Route	Description
POST	/api/auth/register	Créer un compte utilisateur + panier automatique
POST	/api/auth/[...nextauth]	Connexion via NextAuth (credentials JWT)

4.2 Produits

Méthode	Route	Description
GET	/api/products	Récupérer tous les produits (filtre ?category=PELUCHE)

4.3 Panier

Méthode	Route	Description
GET	/api/cart	Récupérer le panier de l'utilisateur connecté
POST	/api/cart	Ajouter un produit au panier (ou incrémenter la quantité)
PATCH	/api/cart/[itemId]	Modifier la quantité d'un article (si quantité < 1 : suppression)
DELETE	/api/cart/[itemId]	Supprimer un article du panier

4.4 Commandes

Méthode	Route	Description
POST	/api/orders	Créer une commande depuis le panier + vider le panier
GET	/api/orders	Récupérer toutes les commandes de l'utilisateur connecté

4.5 Profil

Méthode	Route	Description
GET	/api/profile	Récupérer les infos du profil connecté (sans le mot de passe)
PATCH	/api/profile	Modifier le prénom, l'email ou le mot de passe

4.6 Administration (ADMIN uniquement)

Méthode	Route	Description
GET	/api/admin/products	Lister tous les produits
POST	/api/admin/products	Ajouter un nouveau produit
DELETE	/api/admin/products/[id]	Supprimer un produit (supprime aussi les CartItems liés)
GET	/api/admin/users	Lister tous les clients (role USER) avec leurs commandes

5. Structure du projet

5.1 Arborescence

```
yummyland/
├── app/
│   ├── page.tsx                    # Page d'accueil
│   ├── layout.tsx                  # Layout global (Navbar)
│   ├── globals.css                 # Styles globaux Tailwind
│   ├── shop/page.tsx               # Boutique
│   ├── cart/page.tsx               # Panier
│   ├── orders/page.tsx             # Historique commandes
│   ├── profile/page.tsx            # Profil utilisateur
│   ├── admin/page.tsx              # Panel admin
│   ├── login/page.tsx              # Connexion
│   ├── register/page.tsx           # Inscription
│   └── api/
│       ├── auth/
│       │   ├── [...nextauth]/route.ts
│       │   └── register/route.ts
│       ├── cart/
│       │   ├── route.ts
│       │   └── [itemId]/route.ts
│       ├── orders/route.ts
│       ├── products/route.ts
│       ├── profile/route.ts
│       └── admin/
│           ├── products/
│           │   ├── route.ts
│           │   └── [productId]/route.ts
│           └── users/route.ts
├── components/
│   ├── Navbar.tsx
│   └── SessionProvider.tsx
├── lib/
│   ├── auth.ts                     # Config NextAuth
│   └── prisma.ts                   # Client Prisma singleton
├── prisma/
│   ├── schema.prisma
│   └── seed.ts
├── public/
│   ├── images/                     # Photos produits
│   ├── icons/                      # Icônes navbar/catégories
│   └── logo/                       # Logo YummyLand
├── types/
│   └── next-auth.d.ts
└── .env
```

6. Règles métier

6.1 Inscription

- Mot de passe : minimum 12 caractères, 1 majuscule, 1 minuscule, 1 chiffre, 1 caractère spécial
- Email obligatoirement au format utilisateur@domaine.fr/.com/.net (validation par regex)
- Un panier vide est créé automatiquement à l'inscription
- Les mots de passe sont hashés avec bcrypt (salt 10)

6.2 Panier

- Un seul panier par utilisateur
- Si un produit déjà présent est ajouté, la quantité s'incrémente
- Si la quantité descend à 0 via PATCH, l'article est automatiquement supprimé

6.3 Commande

- Numéro de commande généré automatiquement au format YL-YYYYMMDD-XXXX
- Le panier est vidé automatiquement après validation de la commande
- Le prix des articles est figé au moment de la commande (stocké dans OrderItem.price)
- Statuts possibles : EN_ATTENTE → EN_COURS → LIVREE

6.4 Administration

- Seul un utilisateur avec role = ADMIN peut accéder aux routes /api/admin/*
- Le lien Admin dans la navbar n'est visible que pour les admins
- La suppression d'un produit supprime aussi ses CartItems liés (pas les OrderItems)
- Pour créer un admin : UPDATE User SET role = 'ADMIN' WHERE email = '...';

7. Installation et démarrage

7.1 Prérequis

- Node.js >= 18
- MySQL >= 8 (via Laragon ou autre)
- npm ou yarn

7.2 Étapes

1. Installer les dépendances

```
npm install
```

2. Configurer les variables d'environnement

```
cp .env.example .env
```

Renseigner dans .env :

```
DATABASE_URL="mysql://user:password@localhost:3306/yummyland"
NEXTAUTH_SECRET="chaine_aleatoire_longue"
NEXTAUTH_URL="http://localhost:3000"
```

3. Créer les tables en base

```
npx prisma db push
npx prisma generate
```

4. Peupler la base avec les produits de démonstration

```
npm run prisma:seed
```

5. Lancer le serveur de développement

```
npm run dev
```

L'application est accessible sur http://localhost:3000

7.3 Créer le premier admin

```
UPDATE User SET role = 'ADMIN' WHERE email = 'votre@email.fr';
```

Se déconnecter et se reconnecter pour que le token JWT soit mis à jour.

8. Sécurité

Point de sécurité	Implémentation
Mots de passe	Hashage bcrypt avec salt 10 — jamais stocké en clair
Sessions	JWT signé via NEXTAUTH_SECRET — expire automatiquement
Routes API protégées	getServerSession() vérifie la session côté serveur à chaque appel
Accès admin	Vérification du rôle ADMIN sur chaque route /api/admin/*
Validation email	Regex côté client ET double vérification côté serveur
Validation mot de passe	5 règles vérifiées côté client (12 car, maj, min, chiffre, spécial)
.env	Fichier exclu du dépôt Git via .gitignore